

Hermes: Boosting the Performance of Machine-Learning-Based Intrusion Detection System Through Geometric Feature Learning

Chaoyu Zhang¹, Student Member, IEEE, Shanghao Shi², Member, IEEE, Ning Wang³,
Xiangxiang Xu, Member, IEEE, Shaoyu Li, Student Member, IEEE, Lizhong Zheng⁴, Fellow, IEEE,
Randy Marchany⁵, Member, IEEE, Mark Gardner, Member, IEEE, Y. Thomas Hou⁶, Fellow, IEEE,
and Wenjing Lou⁷, Fellow, IEEE

Abstract—Anomaly-Based Intrusion Detection Systems (IDSs) have been extensively studied for their ability to detect zero-day attacks by establishing a baseline of normal behavior from benign traffic and flagging deviations as potential threats. However, they often suffer from higher false positive rates compared to signature-based IDSs. Unlike image data, raw network traffic lacks directly useful features and requires additional processing to capture meaningful patterns. Simple traffic statistics (e.g., connection duration, packet inter-arrival time) often fail to represent the complex relationships necessary for accurate detection, making feature engineering essential. We propose a geometric feature learning method to optimize feature extraction. Specifically, we employ contrastive feature learning to build a representation space where benign traffic forms a compact cluster, and then apply H-Score feature learning to further enhance intra-class compactness and inter-class separability, thereby improving anomaly detection performance. Beyond the NSL-KDD and N-BaIoT datasets, we further validate our approach on more

advanced and realistic benchmarks, including CIC-IDS-2017, TON-IoT, and UNSW-NB15, demonstrating strong generalizability. To address privacy concerns and enable collaborative learning across distributed environments, we extend our centralized design to a federated learning (FL) setting, preserving user privacy while leveraging the collective knowledge of diverse IoT devices. Finally, we deploy the proposed IDS on a Raspberry Pi 4 to showcase its efficiency and suitability for resource-constrained IoT scenarios.

Index Terms—Machine learning-based intrusion detection system, geometric feature learning, contrastive learning, federated learning, anomaly detection, IoT security.

I. INTRODUCTION

THE development of cloud computing, the expansion of 5G networks, and the exponential growth of IoT devices have led to an unprecedented increase in network traffic and complexity. This expansion not only facilitates innovative applications but also causes significant vulnerabilities [2]. Cyber threats have evolved to be more sophisticated, and attackers are exploiting these vulnerabilities to compromise the confidentiality, integrity, and availability of critical information on the network. The 2016 Mirai botnet [3] is a notable example of powerful distributed denial-of-service (DDoS) attacks. In these attacks, numerous Internet-connected devices, infected with Mirai malware, flooded targeted servers with excessive Internet traffic. This attack rendered major websites, including Twitter, Netflix, Reddit, and CNN, temporarily inaccessible to millions of users. Given these circumstances, the implementation of an intrusion detection system (IDS) has become imperative. IDS plays a crucial role in the continuous monitoring and analysis of network traffic, capable of identifying potential attacks.

In general, there are two types of IDSs to safeguard network infrastructures: signature-based and anomaly-based IDS. Signature-based IDS functions by comparing observed traffic against a database of known attack patterns or signatures [4], [5], [6], [7]. This method is highly effective in recognizing and thwarting established threats, making it an essential tool for defending against known attacks. However, its efficacy is limited to known threats, leaving a gap that anomaly-based IDS aims to fill. Anomaly-based IDS works on the

Received 15 January 2025; revised 11 August 2025 and 5 February 2026; accepted 4 May 2026; approved by IEEE TRANSACTIONS ON NETWORKING Editor E. Bertino. Date of publication 26 May 2026; date of current version 1 June 2026. This work was supported in part by the Office of Naval Research under Grant N00014-24-1-2730 and Grant N00014-19-1-2621; in part by U.S. National Science Foundation under Grant 1916902, Grant 2154929, Grant 2247560, Grant 2235232, and Grant 2312447; and in part by Virginia Commonwealth Cyber Initiative (CCI). The preliminary version of this paper was published in ACM MobiHoc 2024 [DOI: 10.1145/3641512.3686380]. (Corresponding author: Chaoyu Zhang.)

Chaoyu Zhang, Shaoyu Li, and Wenjing Lou are with the Department of Computer Science, Virginia Tech, Arlington, VA 22203 USA (e-mail: chaoyu@vt.edu; shaoyuli@vt.edu; wjlou@vt.edu).

Shanghao Shi is with the Department of Computer Science, Washington University in St. Louis, St. Louis, MO 63144 USA (e-mail: shanghao@wustl.edu).

Ning Wang is with the Department of Computer Science and Engineering, University of South Florida, Tampa, FL 33620 USA (e-mail: ningw@usf.edu).

Xiangxiang Xu is with the Department of Computer Science, University of Rochester, Rochester, NY 14627 USA (e-mail: xiangxiangxu@rochester.edu).

Lizhong Zheng is with the Department of Electrical Engineering and Computer Science, Massachusetts Institute of Technology, Cambridge, MA 02139 USA (e-mail: lizhong@mit.edu).

Randy Marchany is with the IT Security Office, Virginia Tech, Blacksburg, VA 24061 USA (e-mail: marchany@vt.edu).

Mark Gardner is with the Advanced Research Computing, Virginia Tech, Blacksburg, VA 24061 USA (e-mail: mkg@vt.edu).

Y. Thomas Hou is with the Bradley Department of Electrical and Computer Engineering, Virginia Tech, Blacksburg, VA 24061 USA (e-mail: thou@vt.edu).

Digital Object Identifier 10.1109/TON.2026.3697115

principle of behavioral analysis [8], [9], [10], [11], [12], [13]. It establishes a baseline of normal network behavior and continuously monitors for deviations, flagging any unusual activity that could signify a potential intrusion. This approach enables it to detect novel (a.k.a., zero-day) attacks, which do not match any known signatures. While more adaptable to emerging threats, anomaly-based IDS can be prone to higher false-positive rates [14], [15].

Research Motivation: An effective anomaly-based IDS should not only detect intrusions but also identify their types, including both known attacks and generalized zero-day threats. This capability is essential for Intrusion Response Systems (IRS) [16], [17], which rely on attack type identification to trigger targeted responses. For example, a DDoS attack may require rate limiting [18], while malware infiltration may demand system isolation [19], [20]. Detecting zero-day attacks is also critical [21], [22], as it enables the discovery of new signatures to enhance signature-based IDSs and provides novel samples to improve anomaly-based models through retraining.

However, building such an all-in-one solution is particularly challenging for resource-constrained devices, which have limited compute, memory, and energy. Signature-based IDSs require substantial resources to maintain and update attack signature databases, an especially difficult task in large-scale IoT environments with limited bandwidth and massive device populations.

Research Objective: Therefore, the goal of our research is to 1) *improve the overall performance of the anomaly-based IDS by minimizing the false positive rate and reducing the incidence of detection omissions*, and 2) *extend the capabilities of anomaly-based IDS by enabling them not only to detect an intrusion but also to precisely identify the specific type of the intrusion for resource-limited devices*.

In the conventional approach to machine learning-based IDS, feature engineering holds substantial importance within the pipeline as it directly impacts the ultimate detection performance. Unlike image data, where the observed features provide immediate utility, raw network traffic necessitates additional processing for effective detection [23], [24]. Learning useful patterns directly from raw data traffic or simple traffic statistics (e.g., connection duration, packet inter-arrival time) is challenging as the complex relationships are difficult to distinguish. *Therefore, it becomes imperative to extract features that can directly enhance the performance of detection models rather than relying solely on raw data traffic or simple traffic statistics.*

Existing contrastive learning-based IDS methods, such as FeCo [14] and CIDS [25], employ contrastive loss to learn representations of complex network traffic and improve IDS performance. AOC-IDS [26] introduces a contrastive loss tailored for autoencoders to obtain better baseline representations, thereby enhancing anomaly detection.

Our Contribution: Going beyond contrastive learning, which relies on similarity-based objectives to structure representations in latent space, we interpret the neural network's latent space through a geometric lens. We propose Hermes [1], a novel IDS that projects complex network traffic into a geometrically optimized space for enhanced representation, enabling clearer decision boundaries and more accurate intrusion detection. We adopt geometric feature learning [27],

which enables principled, distance-based feature extraction with a clear geometric interpretation. For clarity, we define **features** as the *feature representations extracted by the neural network*. The *space mapped by this neural network* is referred to as the **feature space**, and the *space encompassing all benign feature representations* is termed the **baseline**.

The high-level design of Hermes includes a two-step training phase. The first step involves **Contrastive Feature Learning**. To achieve clear separation between benign and malicious traffic representations in the latent space, we employ it to guide the representation structure. Our approach forms a *positive pair* using two normal traffic records, and a *negative pair* using one normal and one abnormal traffic record. This learning process pulls positive pairs closer and pushes negative pairs farther apart, thereby generating a new baseline. In this space, benign instances are tightly clustered, while attack instances are repelled from the benign cluster, enhancing the effectiveness of subsequent anomaly detection. The second step involves **H-Score Feature Learning**, which further refines the learned features by maximizing the compactness of the cluster representing normal behavior. It also increases the separation between different types of traffic, enabling the IDS to more clearly distinguish between attack classes. The theoretical foundation of the geometrically optimal distance optimization underpinning H-Score feature learning is formally established in [27] and [28].

For IDS inference, Hermes employs a **dual inference** mechanism: it uses a **similarity-based rule** to measure deviations of traffic features from the baseline for anomaly-based detection, and a **Maximum A Posteriori (MAP)** rule for attack type identification. This inference mechanism also evaluates if an instance is a zero-day attack using **entropy-based uncertainty** from the posterior distribution and **inconsistency** between the two predictions.

In this work, we further extend the Hermes centralized design to a decentralized setting through federated learning (FL). In centralized IDSs, while collecting data from multiple sources can provide better intrusion detection performance, they are hindered by privacy concerns and legal restrictions, such as those imposed by HIPAA,¹ which prevent the collection of sensitive user data on central servers. On the other hand, naïve distributed IDSs, trained only with local private datasets, often suffer from low performance due to both the quantity and quality of local data. FL offers a promising solution to these challenges by enabling local gateways to collaboratively train a global model without sharing raw data, thus preserving privacy while leveraging the collective knowledge of diverse IoT devices [29], [30], [31], thereby enabling the creation of more powerful IDS models for various IoT devices. Therefore, we build FL-Hermes by incorporating geometric feature learning into the FL system to achieve better detection performance while simultaneously preserving user data privacy. In this FL-based design, each gateway trains an individual IDS model locally via geometric feature learning, and all gateways cooperatively upload local updates to a central parameter server for model aggregation. After the FL-Hermes training phase, the well-trained model is deployed to these gateways for traffic

¹<https://www.hhs.gov/hipaa/index.html>

TABLE I
A REVIEW OF MACHINE-LEARNING-BASED INTRUSION DETECTION SYSTEMS: MECHANISMS, CAPABILITIES, AND TARGETED THREATS

Name	ML Mechanism	IDS Capability			Security Threats/Source
		Anomaly Detection	Type Identification	Zero-Day Analysis	
Two-Tier [11], 2017	Combination of Neural Networks	✓	✓		DoS, Probing, U2R, R2L [32]
Kitsune [33], 2018	An ensemble of Autoencoders	✓			Recon, Man-in-the-Middle, DoS, and Mirai Botnet [33]
ESFCM [34], 2018	ELM-based semi-supervised Fuzzy C-Means	✓			DoS, Probing, U2R, R2L [32]
ID-CVAE [10], 2017, ICVAE [35], 2019	Conditional Variational Autoencoder	✓			Exploits, Fuzzers, Recon, Analysis, Backdoor, Shellcode, Worms, etc [32], [36]
TDTC [37], 2019	Naive Bayes and K-Nearest Neighbor	✓	✓		DoS, Probing, U2R, R2L [32]
EIDS [38], 2019	Ensemble of Decision Tree, Naive Bayes, and Neural Network	✓			Analysis, Backdoor, DoS, Exploit, Fuzzers, Generic, Recon, etc [36], [39]
AE-DNN [12], 2019	Autoencoder plus Deep Neural Network	✓	✓		Flooding, Impersonation, Injection [40]
RF-SD-IoT [41], 2020	Random Forest	✓	✓		SYN, Ping Flood, UDP Port Scan, UDP Flood [42]
IoTDefender [43], 2020	Convolutional Neural Network	✓	✓		PortScan, DDoS, ARP Spoofing, ARP MitM, Fuzzing, Mirai [32], [44], [33]
SDRK [45], 2020	Deep Neural Network with Random Sampling-K-Means	✓			DoS, Probing, U2R, R2L [32]
Passban [4], 2020	Isolation Forest, Local Outlier Factor	✓	✓		Port Scan, HTTP&SSH Brute Force, SYN Flood [4]
CIDS [25], 2022	Contrastive-Learning	✓	✓		Botnets, Stealthy, Trojans, Ransomware, Exploits, Fuzzers, Recon, Backdoor, Shellcode, Worms [25], [32], [36]
AOC-IDS [26], 2024	Autoencoder Contrastive-Learning	✓	✓		Exploits, Fuzzers, Recon, Analysis, Backdoor, Shellcode, Worms [32], [36]
FeCo [14], 2025	FL Contrastive-Learning	✓	✓		DoS, Probing, U2R, R2L, Mirai, and BashLite [46], [32]
Hermes	Geometric Feature Learning	✓	✓	✓	DoS, Probing, U2R, R2L, Mirai, and BashLite [46], [32]

inference. We further extend our evaluation to demonstrate the effectiveness of Hermes in the FL setting.

Our contributions are summarized as follows:

- 1) Hermes employs contrastive feature learning to establish a well-formed baseline to distinguish benign and intrusion features, thus enabling anomaly detection.
- 2) Building on this, Hermes employs H-Score feature learning to capture the dependencies between traffic and their types, enabling the inference phase to identify the attack types. This method focuses on reducing geometrical distance in the baseline, narrowing distances among features of the same type, and increasing distances between features of different classes, further enhancing anomaly detection ability and enabling attack identification.
- 3) The dual inference of Hermes applies a similarity-based rule to the learned features to determine if the traffic is malicious and utilizes MAP estimation to assess the preliminary type of intrusion. Furthermore, it classifies the types of known attacks and flags zero-day attacks.
- 4) We extend the Hermes centralized design to an FL-based distributed setting to address privacy concerns in centralized IDS learning processes and overcome the performance limitations of naïve distributed IDSs trained on low-quality and low-quantity data.
- 5) Extensive evaluations of Hermes across five datasets and diverse hardware platforms, in both centralized and federated learning settings, have consistently demon-

strated its effectiveness. Hermes outperforms various state-of-the-art IDS baselines.

II. RELATED WORK

A critical factor in anomaly-based IDS performance is the construction of a representative baseline space. Prior work has explored various strategies: Yan et al. [47] employed non-parametric density estimation to learn legitimate access patterns, while KNN-based approaches [11] assess similarity to benign instances. Deep learning models have also been applied, including LSTMs for log modeling [48], autoencoder ensembles for traffic features [33], and VAEs for feature compression [10], [35]. Sequence-based modeling using GRUs [49] and Isolation Forests [41], [43] further contribute diverse anomaly detection strategies. Recent methods such as FeCo [14], CIDS [25], and AOC-IDS [26] leverage contrastive learning to improve feature representations and baseline quality. To enhance intrusion classification, signature-based IDSs [4], [5], [6], [7] match traffic patterns against known attack signatures. For example, graphical models [9], SVMs for botnet detection [8], two-tier classifiers [37], and hybrid clustering-classification approaches [34] demonstrate the strengths of this paradigm in identifying specific attack types. We summarize existing ML-based IDS by analyzing their underlying ML mechanisms, capabilities, and the security threats they target in Tab. I.

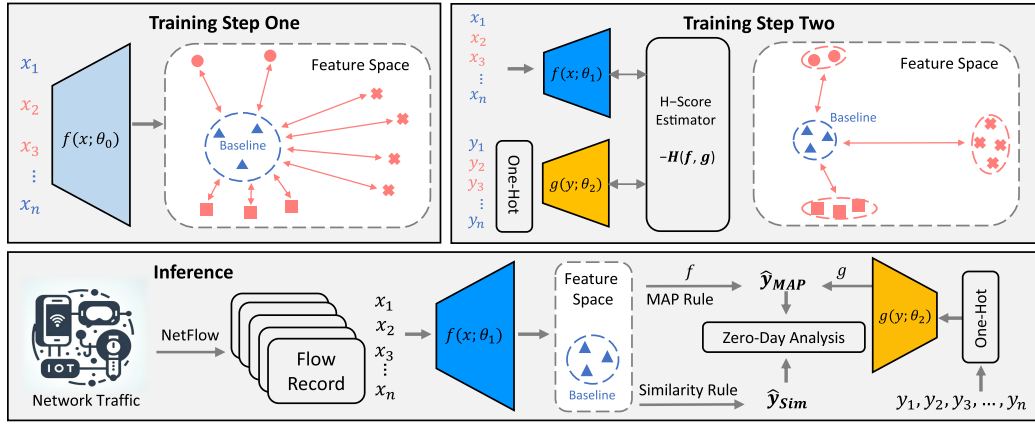


Fig. 2. **Hermes Design:** Our approach begins with contrastive feature learning to train a model $f(x; \theta_0)$, establishing a baseline space (Training Step One). Subsequently, we refine this model to $f(x; \theta_1)$ and develop the identification model $g(y; \theta_2)$ using H-Score feature learning (Training Step Two). During the Inference phase, the established IDS model employs a similarity-based rule for anomaly detection and a MAP rule for attack identification. We further analyze each instance to ascertain whether it could be a zero-day attack, based on the results from dual inference.

two critical steps: first, we construct an initial anomaly detection model $f(x; \theta_0)$ using contrastive feature learning to define a baseline. We then refine this model from $f(x; \theta_0)$ to $f(x; \theta_1)$, and develop the identification model $g(y; \theta_2)$ through H-Score feature learning (as shown in Fig. 2). Once the fully trained IDS model is deployed at the gateway $\mathcal{G}$ (5), Hermes performs dual inference: it utilizes a similarity-based rule for anomaly detection and a MAP rule for attack identification, and analyzes whether the intrusion is a zero-day attack. The system's pivotal elements, contrastive feature learning, H-Score feature learning, and dual inference, are expounded in the following sections.

B. Training Step One: Contrastive Feature Learning

To establish a representative baseline for anomaly detection, we employ geometrical feature extraction through contrastive feature learning. This method leverages the inherent differences and similarities in network traffic to enhance the performance of anomaly-based IDS. Specifically, each training data record in our system consists of two components: the flow record $x \in \mathbb{R}^d$, where d represents the dimensionality of the flow record, and its corresponding output label $y \in \{0, 1\}$. Here, 0 denotes benign traffic flow, while 1 indicates an intrusion. The primary aim of our contrastive learning algorithm is to extract a representative benign feature cluster as a baseline.

To achieve this, we train a neural network $f(x; \theta_0)$ with weights θ_0 , which takes the input record $x \in \mathbb{R}^d$ and produces a feature $f(x) \in \mathbb{R}^k$. Let x_i and x_j represent benign inputs, and x_m represent an intrusive input. Given a training dataset containing benign and intrusion instances, we compute feature representations $f(x_i)$ and $f(x_j)$ for pairs of benign samples. Here, x_j is another benign sample within the same mini-batch as x_i to form a positive pair. Positive and negative pairs are constructed dynamically within each mini-batch to reduce computational overhead, following the strategy in [58].

In contrastive feature learning, training is performed in a mini-batch manner. Within each mini-batch of size B , we denote benign samples as x_i and x_j , where x_i is an anchor benign sample and x_j is another benign sample in the same mini-batch forming a positive pair, and malicious samples as

TABLE II
DEFINITIONS AND NOTATIONS

Symbol	Definition
X	Network flow records variable
Y	Network traffic labels variable
x	An instance of network flow record
y	A label for corresponding flow record
P_X, P_Y	Distributions of X and Y respectively
$P_{X,Y}$	Joint distribution of X and Y
$P_{Y X}$	Posterior distribution given X
$i_{X,Y}$	CDK function, the statistical dependence of X, Y
$\zeta_{\leq k}(i_{X,Y})$	Top k modes of $i_{X,Y}$
$\mathcal{F}_X^k, \mathcal{F}_Y^k$	k -dimensional feature spaces for X and Y
$f \in \mathcal{F}_X^k$	k -dimensional features
$g \in \mathcal{F}_Y^k$	k -dimensional features
$f \otimes g$	Joint function
$\mathcal{F}_{X \times Y}$	Functional space of all joint functions $f \otimes g$
l_{ij}	Pair-wise loss function
$\mathcal{L}$	Cumulative loss function
$\mathcal{H}(f, g)$	H-Score
$\text{tr}(\cdot)$	Trace function, sum of elements on the diagonal
$\mathbb{I}(\cdot)$	Indicator function
$\ \cdot\ _{\mathcal{F}}$	Induced norm in feature geometry, defined as $\ \gamma\ _{\mathcal{F}}^2 = \mathbb{E}_{P_X P_Y} [\gamma^2(X, Y)]$
$f(x; \theta_0)$	Model learned by contrastive feature learning
$f(x; \theta_1)$	Model refined by H-Score feature learning
$g(y; \theta_2)$	Identification model
$f(x; \Theta_0)$	FL local model learned by contrastive feature learning
$f(x; \Theta_1)$	FL local model refined by H-Score feature learning
$g(y; \Theta_2)$	FL local identification model
$\mathcal{M}^{(t)}, \delta_i^{(t)}$	Global model, Model updates by client i (t iteration)
$\mathcal{G}, \mathcal{S}$	Gateway $\mathcal{G}$, Server $\mathcal{S}$
$\mathcal{Z}(P_{Y X})$	Uncertainty from posterior distribution
IDS	Intrusion Detection System
FL	Federated Learning
CL	Contrastive Learning
Sim	Similarity Rule
MAP	Maximum A Posteriori estimation
B	Mini-batch size
$\mathcal{B}$	A mini-batch of flow records
x_i	Anchor benign sample
x_j	Positive benign sample paired with x_i
x_m	Malicious (negative) sample
N	Number of benign samples in a mini-batch
M	Number of malicious samples in a mini-batch

x_m , which serve as negative samples. Let N and M denote the numbers of benign and malicious samples in the mini-batch,

respectively. The values of N and M are not manually specified hyperparameters; instead, they are determined by the natural class distribution of the (non-IID) training dataset within each mini-batch and may vary across batches. Intuitively, a sufficient number of benign samples (N) helps learn a compact and stable representation of normal traffic, while the presence of malicious samples (M) promotes separability between benign and intrusive behaviors. The mini-batch size B controls the trade-off between representation stability and computational cost, and is selected accordingly in our evaluation. Notably, malicious samples are typically not used as anchors because anomaly detection fundamentally relies on modeling consistent benign behavior and detecting deviations from it. Incorporating variable and sparse malicious data as anchors may obscure the learned boundary by introducing unstable or non-generalizable representations. We follow the original dataset's class distribution and a widely used partitioning strategy to preserve the benign-to-malicious ratio. A complete list of definitions and notations is provided in Table II.

Based on the above mini-batch construction and sample roles, the contrastive learning objective aims to maximize the similarity between $f(x_i)$ and $f(x_j)$, while simultaneously distinguishing between benign $f(x_i)$ and all malicious $f(x_m)$ instances, where $f(x_m)_{m \in [M]}$. In this context, $\mathbf{u}_i = f(x_i)$ and $\mathbf{u}_j = f(x_j)$ are the features of benign inputs x_i and x_j , respectively, and $\mathbf{v}_m = f(x_m)$ is the feature of an intrusive input x_m .

$$l_{ij} = -\log \left(\frac{\exp(\mathbf{u}_i^T \mathbf{u}_j / \tau)}{\exp(\mathbf{u}_i^T \mathbf{u}_j / \tau) + \sum_{m=1}^M \exp(\mathbf{u}_i^T \mathbf{v}_m / \tau)} \right). \quad (1)$$

The term $0 < \tau < 1$ represents the temperature coefficient. The overall loss function $\mathcal{L}$ aggregates all pairwise losses, encompassing both l_{ij} and l_{ji} terms:

$$\mathcal{L} = \frac{1}{N(N-1)} \sum_{i=1}^N \sum_{j=i+1}^N (l_{ij} + l_{ji}). \quad (2)$$

Phase one of the training, as depicted in Fig. 2, offers an intuitive visualization of the learning process. In this phase, by minimizing the loss function, we enhance the similarity among benign data representations, effectively drawing them closer together. Concurrently, this process distances benign representations from those indicative of intrusions. Once the model converges, it results in a well-defined baseline for normal traffic, which is essential for effective anomaly detection.

C. Training Step Two: H-Score Feature Learning

While our current model is adept at anomaly detection, it cannot identify specific types of attack and the baseline is not optimal yet. To address this limitation, we employ H-Score feature learning [28], [59], which explores the geometric feature space beyond the established baseline. In essence, H-Score learning geometrically maps the feature to construct unique clusters for each attack type. This enables the identification of specific attack types during the inference phase. Additionally, it increases the separability between features of different classes. This approach further refines the baseline feature space, distancing it from intrusion clusters, which in turn enhances the performance of the anomaly-based IDS.

To begin, we define a variable X indicating flow records, and a variable Y as corresponding labels. In this context, each label y_i is an integer where $y_i \in \{0, 1, \dots, n-1\}$; here, 0 represents a benign label, while any non-zero number corresponds to a specific type of attack. These variables, X and Y , follow the joint distribution $P_{X,Y}$. Given the complexity of traffic encountered by IDS, this joint distribution $P_{X,Y}$ is typically unknown, making the direct computation or estimation of the statistical dependence, as indicated by the CDK (canonical dependence kernel) function $\mathbf{i}_{X,Y}$ (referenced in [27, Eq. 4]), infeasible.

$$\mathbf{i}_{X,Y} = \frac{P_{X,Y}(x,y) - P_X(x)P_Y(y)}{P_X(x)P_Y(y)}. \quad (3)$$

Consequently, we focus on learning $\mathbf{i}_{X,Y}$ from data samples using H-Score feature learning, specifically by considering its rank- k approximation $\zeta_{\leq k}(\mathbf{i}_{X,Y})$, given $k \geq 1$. The approximation of $\mathbf{i}_{X,Y}$ by the rank- k joint function is expressed as $f \otimes g = \sum_{i=1}^k f_i \otimes g_i$, where $f \in \mathcal{F}_X^k$ and $g \in \mathcal{F}_Y^k$ represent k -dimensional features. This formulation translates the computation of $\zeta_{\leq k}(\mathbf{i}_{X,Y})$ into an optimization problem, aiming to minimize the approximation error $\|\mathbf{i}_{X,Y} - f \otimes g\|_{\mathcal{F}}$, where $\|\cdot\|_{\mathcal{F}}$ is as defined in Table II. The optimization variables in this context are the k -dimensional features f and g . However, the error $\|\mathbf{i}_{X,Y} - f \otimes g\|_{\mathcal{F}}$ for a specific f and g cannot be directly computed due to the unknown nature of $\mathbf{i}_{X,Y}$. To overcome this challenge, we utilize the H-Score $\mathcal{H}(f, g)$, defined for $k \geq 1$ and $f \in \mathcal{F}_X^k$, $g \in \mathcal{F}_Y^k$, as

$$\begin{aligned} \mathcal{H}(f, g) &\triangleq \frac{1}{2} \left(\|\mathbf{i}_{X,Y}\|_{\mathcal{F}}^2 - \|\mathbf{i}_{X,Y} - f \otimes g\|_{\mathcal{F}}^2 \right) \\ &= \mathbb{E}[f^T(X)g(Y)] - (\mathbb{E}[f(X)])^T \mathbb{E}[g(Y)] - \frac{1}{2} \cdot \text{tr}(\Lambda_f \Lambda_g), \end{aligned} \quad (4)$$

where $\Lambda_f = \mathbb{E}[f(X)f^T(X)] - \mathbb{E}[f(X)]\mathbb{E}[f(X)]^T$ and $\Lambda_g = \mathbb{E}[g(Y)g^T(Y)] - \mathbb{E}[g(Y)]\mathbb{E}[g(Y)]^T$ are the covariance matrices of $f(X)$ and $g(Y)$ respectively, with $\text{tr}(\cdot)$ denoting the trace function. The H-Score serves as a measure of the quality of the approximation, where a higher H-Score value indicates a lower approximation error. Accordingly, the optimal weights refined through the H-Score are designed to maximize the correlation between the traffic flow record and its specific types. This approach not only maximizes the average distance between different classes but also minimizes the distance within the same class, thereby enhancing class separability. This detailed discussion can be found in [28, Proposition 4], with its proof provided in Appendix F therein.

In the subsequent discussion, we delve into how features are integrated to discern the specific type of traffic. Assuming we have obtained $f \in \mathcal{F}_X^k$ and $g \in \mathcal{F}_Y^k$ by maximizing the H-Score $\mathcal{H}(f, g)$, we determine the traffic flow where $k \geq \text{rank}(\mathbf{i}_{X,Y})$, having established that $f \otimes g = \mathbf{i}_{X,Y}$. With the above assumption, we can obtain the following result, as discussed in [27, Section 3.2].

Suppose $f \otimes g = \mathbf{i}_{X,Y}$. Then, we will have

$$\|\mathbf{i}_{X,Y}\|_{\mathcal{F}}^2 = \text{tr}(\Lambda_f \cdot \Lambda_g), \quad (5)$$

$$P_{Y|X}(y|x) = P_Y(y)(1 + f^T(x)g(y)). \quad (6)$$

Using the features f and g , we are able to compute the strength of the dependence between X and Y , represented

as $i_{X,Y}$. The posterior distribution, as outlined in Eq. 6, is instrumental in identifying the specific type of traffic. In Hermes, where X represents the input records and Y denotes the types, the system obtains the results of the posterior distribution $P_{Y|X}$ of the label Y by Eq. 6. The traffic type is determined through a corresponding maximum a posteriori (MAP) estimation, defined as follows:

$$\hat{y}_{\text{MAP}}(x) = \arg \max_{y \in \mathcal{Y}} P_{Y|X}(y|x) \quad (7)$$

$$= \arg \max_{y \in \mathcal{Y}} P_Y(y)(1 + f^T(x)g(y)), \quad (8)$$

where P_Y can be sourced from the monitored daily traffic. This method is proposed and explained in [27, Eq. 25].

A visual representation of H-Score feature learning is illustrated in Fig. 2. In Training Step Two, the primary loss function used is $-\mathcal{H}(f, g)$. This function is used for calculating the optimal parameters θ_1 and θ_2 during the second step of the training phase. It is specifically designed for the joint training of the parameters θ_1 and θ_2 in a two-network architecture. In this architecture, the neural network $f(x; \theta_1)$ is responsible for generating k -dimensional features f in the feature space $\mathcal{F}_X^k$. The identification model $g(y; \theta_2)$ is a fully connected network, given the underlying distribution of Y . This network takes the one-hot encoded vector $[\mathbb{1}\{y = 1\}, \dots, \mathbb{1}\{y = |n|\}]$ of y as input, and produces the k -dimensional features g from labels in the feature space $\mathcal{F}_Y^k$ as output. Thus, the H-Score estimator evaluates the H-Score between f and g to capture the dependence between traffic and the corresponding type. The main goal during the second step of the training phase is to maximize this H-Score for geometrically optimal representation of features, as depicted in the training step two of Fig. 2.

D. Dual Inference Phase

The inference phase, as shown in Fig. 2, is designed to detect the intrusion, identify the type, and explore zero-day attacks. In this phase, two rules are employed, both utilizing the same features extracted from the model $f(x; \theta_1)$. The first rule, a similarity-based detection rule, measures the deviation of traffic features from the learned baseline, thereby enabling anomaly detection. The second rule, a Maximum A Posteriori rule, identifies the type of attack. It compares the posterior distribution for all types of traffic and outputs the type with the maximum value as the classification result. The dual inference mechanism also evaluates if an instance is a zero-day attack using **entropy-based uncertainty** from the posterior distribution and **inconsistency** between the two predictions.

1) *Rule 1: Similarity-Based Detection:* The anomaly-based detection in our system utilizes a threshold-based mechanism based on similarity measures. This process starts with the aggregation of features from the baseline space. We then compute the mean of these normalized features to establish an anchor baseline template, denoted as $\bar{z}$:

$$\bar{z} = \frac{1}{\sum_i \mathbb{1}(y_i = 0)} \sum_i \left(\mathbb{1}(y_i = 0) \frac{f(x_i)}{\|f(x_i)\|_2} \right), \quad (9)$$

where $\mathbb{1}(\cdot)$ denotes the indicator function, with $y_i = 0$ signifying benign traffic, and $\|\cdot\|_2$ represents the L^2 -norm.

To assess the similarity $S(x_i)$ between a new traffic flow x_i and the anchor baseline template $\bar{z}$, we employ the similarity estimator:

$$S(x_i) = \frac{\bar{z}^T f(x_i)}{\|\bar{z}\| \times \|f(x_i)\|}. \quad (10)$$

This similarity score ranges between 0 and 1. In order to classify a new traffic flow x_i as normal or anomalous, we set a threshold score ρ within the range of $[0, 1]$. This threshold is derived from the distribution of similarity scores of the benign training data. We start by sorting these scores in ascending order, resulting in a sequence $\mathbf{S} = [S_1, S_2, \dots, S_N]$. The threshold ρ is then determined by selecting the p -th percentile of this sequence:

$$r = \left\lfloor \frac{p}{100} \cdot N \right\rfloor, \quad (11)$$

where $\lfloor \cdot \rfloor$ denotes the floor function, and p represents a percentage. As a result, the threshold ρ is set to S_r , which corresponds to the r -th value in the sorted sequence $\mathbf{S}$. To minimize the False Positive Rate, a low percentile p is recommended. An observation x is predicted as an intrusion if its similarity score falls below this threshold ρ :

$$\hat{y}_{\text{sim}}(x) = \mathbb{1}(S(x) < \rho). \quad (12)$$

2) *Rule 2: Maximum A Posteriori Rule for Attack Identification:* The second rule, a MAP rule, can help to identify the type of attack. It does this by comparing the correlation between the features extracted from the traffic and the geometric features corresponding to specific attack types. The neural network $f(x; \theta_1)$ processes these records to extract the corresponding features, represented as $f(x)$. The identification model $g(y; \theta_2)$ generates the feature for each class, denoted as $g(y)$. For a new record x , we predict the corresponding attack type using the MAP rule as:

$$\hat{y}_{\text{MAP}}(x) = \arg \max_{y \in \mathcal{Y}} \{P_Y(y)[1 + f^T(x)g(y)]\}, \quad (13)$$

where $P_Y(y)$ is the true traffic type distribution that can be monitored from the daily network flow traffic.

E. Zero-Day Attack Analysis

The dual inference of Hermes is adept at discerning whether an attack could be a zero-day attack.

Suppose an instance is a zero-day attack; it will exhibit new characteristics and previously unseen signatures, leading to increased uncertainty in the model's predictions. This uncertainty is evaluated based on the entropy of the posterior distribution (see Equation 6): $P_{Y|X}(y|x) = P_Y(y) \cdot (1 + f^T(x)g(y))$. The features of a zero-day attack may manifest in a feature space distinct from any known attack's space, resulting in no dominant probability for any known attack in the posterior distribution and consequently, a higher entropy. This metric can help to determine if it is a zero-day attack. We define this uncertainty as follows:

$$\mathcal{Z}(P_{Y|X}(y|x)) = - \sum_i P_{Y|X}(y_i|x) \log P_{Y|X}(y_i|x). \quad (14)$$

Consistent results from similarity-based and MAP rules are typically observed in detection scenarios. However, inconsistencies may arise, such as when the similarity rule indicates an instance is benign, while the MAP rule identifies it as an

attack, or vice versa. These inconsistencies are more likely to occur when the features of a zero-day attack are geometrically situated in the mid-area between the feature spaces of different types of traffic. Therefore, if an instance is a zero-day attack, it is more likely to provoke inconsistent predictions between the similarity and MAP rules, necessitating further investigation to confirm if it is a zero-day attack. We have validated these findings through observations and experimental results.

Algorithm 1 Training Phase

```

1: Training Step One: Contrastive Feature Learning
2: Input: Flow Records with benign inputs  $\{x_i\}_{i=1}^N$  and malicious inputs  $\{x_m\}_{m=1}^M$ 
3: Output: Optimized parameters  $\theta_0$  for  $f(x; \theta_0)$ 
4: for each epoch in num_epochs do
5:   for each benign input pair  $(x_i, x_j)$  do
6:      $\mathbf{u}_i \leftarrow f(x_i), \mathbf{u}_j \leftarrow f(x_j)$ 
7:      $\mathbf{v}_m \leftarrow f(x_m)$ 
8:     /* Compute the pairwise loss  $l_{ij}$  */
9:      $l_{ij} = -\log \left( \frac{\exp(\mathbf{u}_i^T \mathbf{u}_j / \tau)}{\exp(\mathbf{u}_i^T \mathbf{u}_j / \tau) + \sum_{m=1}^M \exp(\mathbf{u}_i^T \mathbf{v}_m / \tau)} \right)$ 
10:   end for
11:   /* Compute the cumulative loss function  $\mathcal{L}$  */
12:    $\mathcal{L} = \frac{1}{N(N-1)} \sum_{i=1}^N \sum_{j=i+1}^N (l_{ij} + l_{ji})$ 
13:   /* Update the parameters */
14:    $\theta_0 = \theta_0 - \alpha \nabla \mathcal{L}$ 
15: end for
16: Training Step Two: H-Score Feature Learning
17: Input: Flow Records with labels  $\{(x_i, y_i)\}_{i=1}^N$ 
18: Output: Optimized parameters  $\theta_1$  for  $f(x; \theta_1)$ , and  $\theta_2$  for  $g(y; \theta_2)$ 
19: for each epoch in num_epochs do
20:   for each  $(x_i, y_i)$  in Training data  $\{(x_i, y_i)\}_{i=1}^N$  do
21:     /* Convert labels  $y$  to one-hot encoding  $y_{\text{one-hot}}$  */
22:      $y_{\text{one-hot}}[i] = \begin{cases} 1 & \text{if } i = y \\ 0 & \text{otherwise} \end{cases}$ 
23:     /* Get  $f$  and  $g$  values using  $x$  and  $y_{\text{one-hot}}$  */
24:      $f = \{f(x_i)\}_{i=1}^N, g = \{g(y_{\text{one-hot}, i})\}_{i=1}^N$ 
25:     /* Compute the negative H-Score as loss,  $\mathcal{L}$  */
26:      $\mathcal{L} = -\mathcal{H}(f, g)$ 
27:     /* Update the models' parameters */
28:      $\theta_1 = \theta_1 - \alpha \nabla \mathcal{L}, \theta_2 = \theta_2 - \alpha \nabla \mathcal{L}$ 
29:   end for
30: end for

```

We summarize the whole training phase and dual inference procedures in Algorithms 1 and 2, and compose the notations in Tab. II. The whole illustration for H-Score feature learning can be found in [27].

F. FL-Hermes Design

In FL-Hermes (Fig. 1 (b)), the learning process, instead of operating on a central server, distributes the training process across a set of local gateways $\mathbf{G} = \{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_N\}$ and utilizes a parameter server $\mathcal{S}$ to coordinate the training. This FL training paradigm allows participants to jointly train a shared IDS model without sharing raw data. The learning process can be outlined as follows:

Algorithm 2 Dual Inference

```

1: Input: Flow Records  $\{(x_i)\}_{i=1}^N$ 
2: Output: Anomaly detection results:  $\{\hat{y}_{Sim}(x_i)\}_{i=1}^N$ , and the traffic types  $\{\hat{y}_{MAP}(x_i)\}_{i=1}^N$ .
3: /* Compute normalized features for benign traffic */
4:  $z_{sum} \leftarrow 0$ 
5: for all benign records  $x_i$  do
6:    $z_{sum} \leftarrow z_{sum} + \frac{f(x_i)}{\|f(x_i)\|_2}$ 
7: end for
8:  $\bar{z} \leftarrow \frac{z_{sum}}{\text{number of benign records}}$ 
9: /* Compute the threshold  $\rho$  from the benign similarity distribution */
10:  $\mathbf{S} \leftarrow$  sorted similarity scores of benign records
11:  $r \leftarrow \lfloor \frac{\rho}{100} \cdot N \rfloor$ 
12:  $\rho \leftarrow S_r$ 
13: for each  $x_i$  in Flow Records  $\{(x_i)\}_{i=1}^N$  do
14:   /* Compute the similarity  $S$  for new traffic flow  $x_i$  */
15:    $S(x_i) \leftarrow \frac{\bar{z}^T f(x_i)}{\|\bar{z}\| \times \|f(x_i)\|}$ 
16:   /* Predict new observation based on similarity score and threshold */
17:    $\hat{y}_{Sim}(x_i) = \mathbb{1}(S(x_i) < \rho)$ 
18:   /* Predict the traffic by MAP rule  $\hat{y}(x)$  */
19:   Estimate the prior distribution  $P_Y(y)$ 
20:    $\hat{y}_{MAP}(x_i) = \arg \max_{y \in \mathcal{Y}} P_Y(y) \cdot (1 + f^T(x_i)g(y))$ 
21:   /* Zero-Day Attack Analysis */
22:    $\mathcal{Z}(P_{Y|X}) = -\sum_j P_{Y|X}(y_j|x_i) \log P_{Y|X}(y_j|x_i)$ 
23:   if  $\mathcal{Z}(P_{Y|X})$  is an outlier then
24:     return zero-day attack
25:   else if  $\hat{y}_{MAP}(x)$  is consistent with  $\hat{y}_{Sim}(x)$  then
26:     return not zero-day attack
27:   else if  $\hat{y}_{MAP}(x)$  is inconsistent with  $\hat{y}_{Sim}(x)$  then
28:     return zero-day attack
29:   end if
30: end for

```

In step ①, at the beginning of FL iteration t , the parameter server $\mathcal{S}$ initializes a global IDS model $\mathcal{M}^{(t)}$. The model architecture is the same as the centralized setting ($f(x; \theta_0)$ and $g(y; \theta_2)$), consistent across all participating local gateways. During each FL iteration t , $\mathcal{S}$ selects a subset of gateways $\mathbf{G}' = \{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_n\} \subseteq \mathbf{G}$ to participate. The global model $\mathcal{M}^{(t)}$ is broadcast to the selected gateways $\mathbf{G}'$.

In the local training phase, each selected gateway $\mathcal{G}_i$, where $\mathcal{G}_i \in \mathbf{G}'$, initializes its local model using the received global model $\mathcal{M}^{(t)}$. The gateway trains the local IDS model using flow records collected from its network, as in step ②. Similarly to the centralized setting, step ③ involves two critical sub-steps: first, the client constructs a local anomaly detection model $f(x; \Theta_0)$ using contrastive feature learning to define a baseline; it then optimizes the traffic representation of this model from $f(x; \Theta_0)$ to $f(x; \Theta_1)$ and develops the identification model $g(y; \Theta_2)$ using H-Score feature learning, allowing the identification of intrusion types and the analysis of zero-day attacks. After training for several local epochs, the gateway computes the model update $\delta_i^{(t)}$ as $\delta_i^{(t)} = \{\Theta_1, \Theta_2\}$. In step ④, Model Update Aggregation, the local gateways upload their model updates $\delta_i^{(t)}$ to the parameter server $\mathcal{S}$. The server aggregates these updates using an aggregation rule, such as

TABLE III
PERFORMANCE (%) OF HERMES ON NSL-KDD DATASET

Method	Acc	Recall	Precis	F1	FPR
Hermes_Sim	90.17	85.55	96.80	90.83	3.74
CL	87.40	81.89	95.31	88.09	5.32
MLP	85.22	77.92	95.29	85.68	5.10
IsoForest	79.54	69.35	92.90	79.42	7.00
SVM	77.16	72.80	84.92	78.39	17.08
VAE	78.33	85.50	77.88	81.96	31.46
LGR	77.41	72.24	85.83	78.45	15.76
BNB	78.43	65.66	94.88	77.61	4.69
KNN	78.50	64.37	96.79	77.32	2.82
DTC	79.09	74.60	86.81	80.24	14.98
AOC-IDS	89.51	96.59	86.54	91.29	19.84
FeCo	89.55	86.80	94.39	90.44	6.82
CIDS	82.29	77.18	93.54	80.65	-
ESFCM	80.69	80.72	80.85	80.45	-
Two-Tier	-	82.00	-	-	5.43
TDTC	-	84.86	-	-	4.86

Hermes_MAP	Recall	Precis	F1	FPR	Ratio
Normal	88.96	92.66	90.77	7.18	50.48
DoS	84.12	99.79	91.29	0.09	35.86
Probe	92.42	91.62	92.02	0.89	9.56
R2L	94.98	56.04	70.49	3.06	3.96
L2R	66.67	1.74	3.38	4.85	0.13
Weighted Avg	87.77	93.55	90.16	3.87	-

Note: Acc: 87.77 is a multi-class accuracy.

FedAvg, to produce a new global model $\mathcal{M}^{(t+1)}$ for the next FL iteration: $\mathcal{M}^{(t+1)} \leftarrow \frac{1}{|\mathcal{G}|} \sum_{i \in \mathcal{G}} \delta_i^{(t)}$. The updated global model $\mathcal{M}^{(t+1)}$ is broadcast back to the selected gateways in the subsequent iteration. This process repeats across multiple FL iterations until the global model converges to an optimal solution $\mathcal{M}^*$, which generalizes well across all gateways. Once the fully trained IDS model $\mathcal{M}^*$ is deployed at the gateway $\mathcal{G}$, it operates, similar to the previous design, as described in step ⑤. The model uses a similarity-based rule to detect anomalies and a MAP rule to identify the type of attack. Additionally, it analyzes whether the detected intrusion is a zero-day attack. This FL-Hermes design ensures data privacy by keeping sensitive flow records locally while leveraging collaborative learning to enhance IDS performance across diverse network environments.

V. EXPERIMENT

We evaluate Hermes under both centralized and federated settings, focusing on three aspects: anomaly detection accuracy, attack identification capability, and performance under zero-day attack scenarios. We further benchmark Hermes against representative baseline methods and state-of-the-art anomaly-based IDSs.

A. Datasets and Experiment Settings

We prototype Hermes using the PyTorch framework and evaluate on two prominent network traffic datasets: NSL-KDD [32] and N-BaIoT [46]. The NSL-KDD dataset, a well-established benchmark in IDS performance evaluation [34], [37], [50], includes benign traffic along with four categories of intrusions, as listed in Tab. IV. We highlight the 16 intrusion sub-classes that are only present in the test dataset. This presents a significant challenge to Hermes in detecting emerging zero-day threats. Each record in the dataset features 41 attributes extracted from network traffic. These

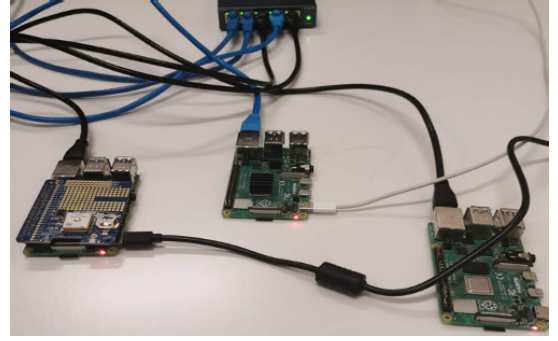


Fig. 3. Evaluating Hermes on Raspberry Pi 4, an IoT device in a resource-constrained environment.

evaluations are conducted on a server equipped with an Intel Core i9-11900K CPU (8 cores / 16 threads, 3.50 GHz base), an NVIDIA GeForce RTX 3080 GPU, and running Ubuntu 20.04.5 LTS.

The N-BaIoT dataset focuses on IoT traffic, encompassing data from nine commercial IoT devices, including those compromised by malicious software, such as Mirai and BASH-LITE. This dataset records benign traffic and five types of attacks for both Mirai ('scan', 'ack', 'syn', 'udp', and 'udplain') and BASHLITE ('scan', 'junk', 'udp', 'tcp', and 'combo') [46]. A principal goal to use this dataset is to verify the applicability of Hermes in resource-constrained IoT environments. Accordingly, our experiments with the N-BaIoT dataset are conducted on a Raspberry Pi 4 as depicted in Fig. 3, equipped with a 1.5 GHz quad-core A72 64-bit ARMv8 CPU, 4GB RAM, 32GB SD card storage, and running the Raspbian OS, to evaluate Hermes's efficiency in such settings.

The default configurations of Hermes are outlined as follows. For the NSL-KDD dataset, the network $f(x; \theta_0)/f(x; \theta_1)$ is composed of an input layer with $d = 112$ dimensions and an output layer with $k = 256$ dimensions. Additionally, it includes two hidden layers with sizes of 128 and 256. The $g(y; \theta_2)$ network, designed for attack identification, utilizes a three-layer structure with 256, 128, and 5 neurons. For the N-BaIoT dataset, the architecture largely mirrors that of the NSL-KDD. The key difference is in the input and output layers of the $f(x; \theta_0)/f(x; \theta_1)$ network, which are configured to accommodate 115 feature inputs and 11 class outputs. The ReLU (Rectified Linear Unit) activation function is implemented after each hidden layer in both models to introduce non-linearity. Hermes employs the Adam optimizer with parameters that include a learning rate of $\alpha = 10^{-4}$ and a mini-batch size of $B = 128$. The training regimen varies between datasets: for the first phase, $e = 20$ epochs are used for NSL-KDD and $e = 15$ for the N-BaIoT dataset. In the second training phase, an additional 20 epochs are applied to both datasets. For comprehensive analysis, Hermes's performance is benchmarked against machine learning models from the renowned scikit-learn library [60].

B. Evaluation Metric and Baselines

We evaluate intrusion detection performance using standard metrics, including Accuracy, Recall, Precision, F1-score, and False Positive Rate (FPR), following commonly adopted definitions in prior IDS studies [11], [34], [37]. In addition,

TABLE IV
INTRUSION CLASSES (SUB-CLASSES) OF THE NSL-KDD DATASET

Category	Attacks sub-classes (Training and Testing Dataset)	Zero-Day Attack sub-classes (Testing Dataset Only)
DoS (10)	back, land, Neptune, pod, smurf, teardrop, udpstorm	apache2, mailbomb, processtable
Probe (6)	ipsweep, nmap, portsweep, satan	mscan, saint
R2L (16)	ftp_write, guesspasswd, imap, multihop, phf, warezmaster, spy, warezclient	httptunnel, named, sendmail, snmpgetattack, snmpguess, xlock, xsnoop, worm
U2R (7)	bufferoverflow, loadmodule, perl, rootkit	ps, sqlattack, xterm

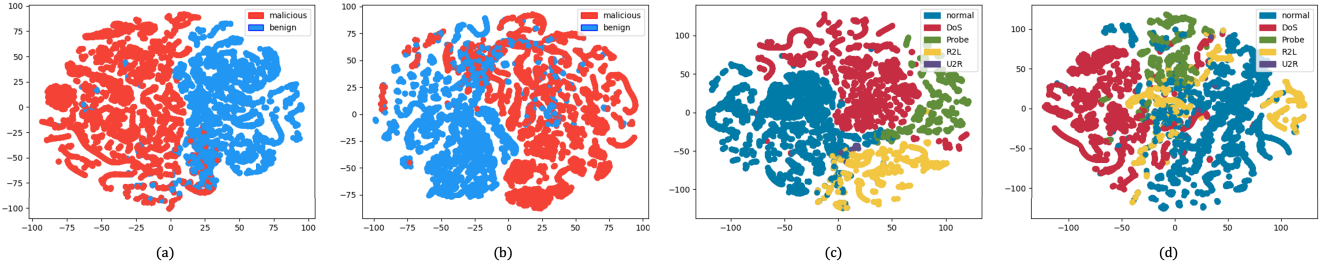


Fig. 4. Visualization of $\mathcal{F}_{\mathcal{X}}^k$ and MLP Output on a Randomly Selected 5% Subset of the NSL-KDD Test Dataset for: (a) Hermes Anomaly-Detection, (b) MLP Anomaly-Detection, (c) Hermes Attack Identification, and (d) MLP Attack Identification.

we report ROC curves by plotting Recall against FPR under varying decision thresholds, and use the Area Under the Curve (AUC) to summarize overall detection performance.

Our experimental configuration is as follows: *Hermes_Sim* employs $f(x; \theta_1)$ with the similarity rule for anomaly detection. *Hermes_MAP* uses $f(x; \theta_1)$ and $g(y; \theta_2)$ with the MAP rule for attack identification. CL (Contrastive Learning) utilizes the architecture of $f(x; \theta_1)$ and adheres to the similarity rule. The multi-layer perceptron (MLP) model integrates the architectures of $f(x; \theta_1)$ and a classification head, optimized using cross-entropy loss. Comparative performance evaluations include models such as support vector machine (SVM), variational autoencoder (VAE), isolation forest (IsoForest), logistic regression (LGR), Bernoulli naive Bayes (BNB), K-nearest neighbors (KNN), and decision tree classifier (DTC), along with IDS methods TDTC [37], Two-Tier [11], ESFCM [34], and contrastive-learning-based variants AOC-IDS [26], FeCo [14], and CIDS [25].

C. Hermes Results on NSL-KDD Dataset

1) *Feature Space Insights*: We utilize t-SNE—a dimensionality reduction technique—to project the feature space, $\mathcal{F}_{\mathcal{X}}^k$, into a two-dimensional figure as shown in Fig. 4. In the presented visualizations of $\mathcal{F}_{\mathcal{X}}^k$ with a randomly selected 5% subset of the NSL-KDD test dataset, distinct patterns emerge when comparing Hermes and the MLP method across both binary anomaly detection and multi-class attack identification. For anomaly detection, the Hermes approach in (a) exhibits a more discernible separation between the “malicious” and “benign” clusters, whereas the MLP model in (b) shows a considerable overlap between the two categories. (c) Hermes and (d) MLP demonstrate multi-cluster separations. Hermes in (c) maintains better-delineated boundaries between the classes, especially when differentiating subtle distinctions like between “DoS”, “Probe”, “U2R”, and “R2L”. In contrast, the MLP’s features in (d) show more intermingling among classes.

2) *Centralized Hermes Evaluation*: In Tab. III, *Hermes_Sim* achieves high accuracy, recall, precision, and F1

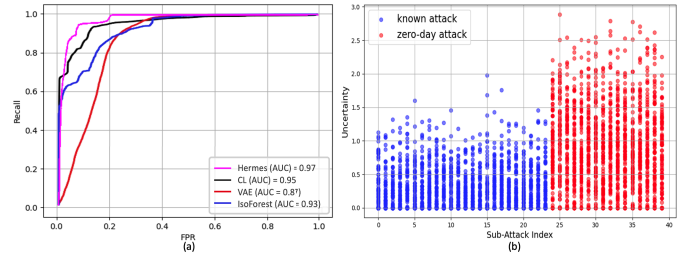


Fig. 5. (a) The ROC curve of *Hermes_Sim*. (b) Uncertainty analysis for traffic subclasses in exploring zero-day attacks.

score among all tested methods while maintaining an FPR as low as 3.74%.

The **ablation study** demonstrates that adding contrastive feature learning to the basic MLP method results in improvements across all metrics, including a 3.97% increase in recall, while keeping a similar FPR. Introducing H-Score feature learning further enhances performance, with overall improvements of 7.63% in recall, while reducing the FPR by 1.36% compared to the baseline MLP. **Detection Speed**: The average inference time (over 100 records) is 3.58 ms for MLP, 3.20 ms for CL, and 4.25 ms for *Hermes_Sim*. Other methods like KNN, although having a low FPR, do not perform as well in Recall, accuracy, and F1 score. TDTC (4.86%) and Two-Tier (5.43%) systems show competitive FPRs but are less effective in other metrics and in identifying specific attack types.

The ROC curves of Hermes and three other baselines are plotted in Fig. 5 (a), with our method exhibiting a higher AUC score compared to the baselines. For intrusion identification, Tab. III shows that *Hermes_MAP* attains an 87.77% accuracy in identifying four classes of intrusions amid normal traffic. The table’s lower section details the identification performance for each traffic class, alongside their ratio in the training data. With adequate samples, Hermes consistently maintains high accuracy. Remarkably, it effectively identifies the R2L attack, which represents a mere 0.13% of the training dataset, with a low FPR of 4.85%.

TABLE V
PERFORMANCE (%) OF HERMES IN FL, NAÏVE DISTRIBUTED LEARNING, AND CENTRALIZED LEARNING

Learning Type	Data Distribution	Acc	Recall	Precis	F1	FPR	DoS	Probe	R2L	U2R
FL	IID	86.64	80.72	93.49	86.64	6.51	80.78	95.19	70.27	60.86
	non-IID-1	87.28	81.60	94.01	87.33	6.07	80.65	91.35	79.02	60.46
	non-IID-2	83.19	73.75	95.02	83.04	4.88	69.52	90.39	69.91	62.03
Naïve Distributed	IID	82.81	74.49	93.05	82.75	6.89	74.57	88.16	52.33	55.42
	non-IID-1	81.67	74.14	93.05	82.52	7.78	77.79	81.26	53.46	61.21
	non-IID-2	67.11	48.68	93.80	64.09	4.89	38.07	39.29	52.69	48.46
Centralized	-	90.17	85.55	96.80	90.83	3.74	84.12	92.42	94.98	66.67

Note: DoS, Probe, R2L, U2R are the Recalls for these specific classes of intrusion.

TABLE VI
DEVICES SPECIFICATION IN N-BAIoT DATASET

Index	Devices Make and Model	Device Type
0	Danmini	Doorbell
1	Ennio	Doorbell
2	Ecobee	Thermostat
3	Philips B120N/10	Baby monitor
4	Provision PT-737E	Security camera
5	Provision PT-838	Security camera
6	SimpleHome XCS7-1002-WHT	Security camera
7	SimpleHome XCS7-1003-WHT	Security camera
8	Samsung SNH 1011 N	Webcam

TABLE VII
IMPACT OF THE NUMBER OF FL CLIENTS ON DETECTION PERFORMANCE

# Clients	Acc	Recall	Precision	F1	FPR
25	84.52	77.86	92.78	84.45	7.34
50	86.64	80.72	93.49	86.64	6.51
100	86.98	81.25	93.66	86.95	6.32
Centralized	90.17	85.55	96.80	90.83	3.74

To illustrate Hermes's ability to identify zero-day attacks, we display in Fig. 5 (b) the uncertainty for each intrusion subclass and normal traffic, calculated using Eq. 14. The x-axis in the figure represents the intrusion subclasses enumerated in Tab. IV, with indices 0 to 23 denoting benign and known attacks, and 24 to 39 indicating zero-day attacks. The y-axis is the uncertainty associated with the corresponding traffic. Notably, known traffic types exhibit lower mean and variance in uncertainty, whereas zero-day samples display significantly higher variance and mean. This pattern suggests that high-uncertainty outliers could be potential zero-day attack samples warranting further investigation. Furthermore, by analyzing the discrepancies between outputs from $\hat{y}_{sim}(x)$ and $\hat{y}_{MAP}(x)$, we find that 77.24% of the intrusions are zero-day attacks. This insight offers an additional method for identifying potential zero-day attack samples.

FL System Setting: We simulate an FL system with 50 clients to evaluate Hermes in distributed settings. This choice follows common practice in prior FL studies (e.g., FeCo [14], PyramidFL [61], AUCTION [62]). We additionally evaluate different client scales by varying the number of clients from 25 to 100 under the same training configuration. As shown in Table VII, FL-Hermes exhibits stable performance across these settings; therefore, we adopt 50 clients in the rest of the evaluation for fair comparison and engineering practicality.

We consider both IID and non-IID data partitioning strategies, where the same training dataset used in the centralized setting is partitioned and distributed among clients. For the

IID data distribution, the training dataset is randomly and evenly divided among all clients, ensuring that each client receives a balanced subset. In the non-IID-1 setting, each client is assigned data from $n \in \{1, 2, 3, 4\}$, representing one to four of the total intrusion attack categories. In the non-IID-2 setting, each client is restricted to data from only one ($n = 1$) intrusion attack category. Notably, the non-IID-2 setting represents a more extreme case of non-IID-1, where FL training becomes significantly more challenging. This is due to the highly uneven data distribution, with each client holding data from only a single intrusion category. The lack of diversity in local datasets amplifies the non-IID nature, resulting in more biased local models and making it more challenging to achieve well-trained global models.

To ensure a fair comparison, the testing dataset across all settings remains the same as in the centralized setting. We evaluate Hermes under three different learning frameworks: FL, naïve distributed learning, and centralized learning. In the naïve distributed learning framework, training is performed entirely on local devices using only local data, and evaluation metrics are averaged across all clients. Centralized learning, as discussed in Sec. V-C2, involves training on a single merged dataset.

For local training, we configure $E = 3$ epochs for both the contrastive feature learning and H-Score feature learning phases. The FL process is set to run for $I = 20$ iterations to train the local models. Across all settings, we use the Adam optimizer with a consistent learning rate to ensure uniformity in the experiments.

Experimental Results: We evaluate the performance of FL-Hermes, naïve distributed learning with Hermes, and centralized Hermes under various local client data distributions, as shown in Tab. V.

FL-Hermes consistently outperforms naïve distributed learning across most evaluation metrics, regardless of the data distribution. Moreover, FL-Hermes demonstrates consistent detection performance across the three data distributions (IID, non-IID-1, and non-IID-2), highlighting the ability of the FL design to enable distributed clients, each holding different types of data, to collaboratively train a robust model. This aligns well with real-world privacy-sensitive application scenarios, where different clients hold their own private data, resulting in inherently highly non-IID distributions. In contrast, naïve distributed learning, although preserving privacy, heavily relies on the quality of local data. For example, under the non-IID-2 distribution, naïve distributed learning achieves an average accuracy as low as 67.11% and an average recall as low as 48.68%, highlighting its vulnerability to imbalanced data distribution and limited local data diversity. While

the detection accuracy of FL-Hermes is lower than that of centralized learning, the latter is challenging to deploy in privacy-sensitive scenarios, making FL-Hermes a more practical alternative.

We also compare the performance of Hermes across different data distributions within the same learning framework. In the FL setting, Hermes achieves an accuracy of 87.28% under the non-IID-1 distribution, which is higher than the 83.19% accuracy achieved under the non-IID-2 distribution. A similar trend is observed for recall, where Hermes achieves 81.60% under the non-IID-1 distribution, compared to 73.75% under the non-IID-2 distribution. Similar trends are also observed in naïve distributed learning across non-IID-1 and non-IID-2 distributions. This decreased diversity makes it more challenging to learn stable representations of benign instances, as geometric feature learning iteratively optimizes the feature representations away from diverse types of attack representations.

In summary, FL-Hermes achieves robust performance across diverse data distributions and significantly outperforms naïve distributed learning. Although centralized Hermes achieves higher accuracy, its practical limitations in privacy-sensitive applications underscore the importance of FL-Hermes as a viable and effective alternative.

D. Hermes Results on N-BaIoT Dataset

1) *IoT Device Evaluation:* To demonstrate the feasibility of utilizing IoT devices for intrusion detection, we conduct evaluations of Hermes on real IoT botnet traffic datasets derived from nine commercially available IoT devices infected by the Mirai and BASHLITE botnets. The specifications of these devices are detailed in Tab. VI, which provides a comprehensive breakdown of their characteristics. For a detailed analysis, we create a specific model for each IoT device and evaluate its performance individually. To simulate a typical resource-constrained IoT environment, we implement our system on a Raspberry Pi 4, as illustrated in Fig. 3.

The Hermes_Sim model consistently demonstrates a lower FPR compared to the CL approach, confirming its effectiveness in reducing erroneous anomaly detections. Meanwhile, the Hermes_MAP variant excels in attack identification tasks, achieving consistently high classification accuracy while maintaining competitive FPR values across all devices. These experimental results highlight the robustness of Hermes in detecting anomalies, identifying attack types, and addressing zero-day attacks. Importantly, the results validate the practicality of deploying Hermes in resource-constrained environments, showcasing its ability to operate effectively across a variety of IoT devices and multi-application scenarios.

To further evaluate the effectiveness of our approach, we compare the performance of Hermes_Sim with several baseline methods across multiple performance metrics, as shown in Fig. 6. Both Hermes_Sim and CL demonstrate high levels of performance across all metrics, including accuracy, recall, precision, and F1 scores, while maintaining consistently low FPR values. These comparisons underscore the superior capability of Hermes_Sim in balancing robust detection performance with minimal false alarms.

2) *FL-Hermes Evaluation:* We extend the evaluation by leveraging FL to collaboratively train a global model across

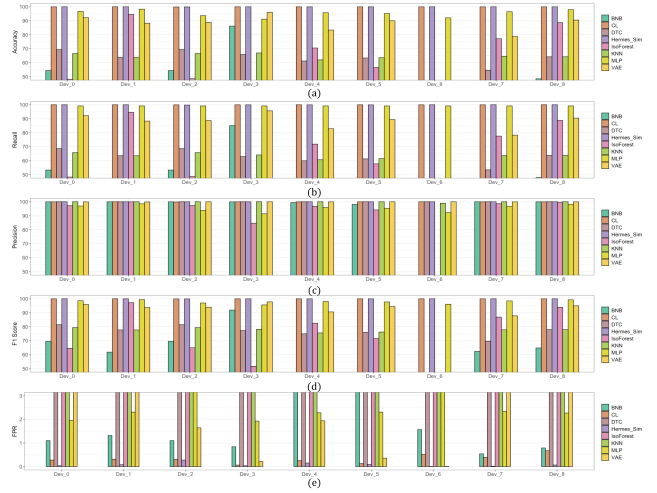


Fig. 6. The performance (%) of Hermes compared with baselines on the N-BaIoT dataset is presented across various metrics: (a) Accuracy, (b) Recall, (c) Precision, (d) F1 Score, and (e) FPR.

IoT devices, instead of training models independently on local data from each device. In this setup, local models are trained by each client and then uploaded to a parameter server for aggregation into a global model. After the FL training phase, the well-trained global model is deployed back to each IoT device, where it is evaluated using the same testing data collected from the respective devices. The results are summarized in Tab. IX.

The experimental results demonstrate the robust performance of FL-Hermes across various evaluation metrics. Specifically, under the similarity-based rule (FL-Hermes_Sim), the system achieves high detection accuracy, recall, precision, and F1 score, all exceeding 99.30%. Additionally, the FPR remains exceptionally low, never exceeding 0.35%. This highlights the system's ability to detect intrusions effectively while minimizing false alarms in a federated environment. Similarly, FL-Hermes MAP Rule achieves consistently strong detection performance across individual IoT devices, even with distributed and heterogeneous local clients.

The ability of FL-Hermes to collaboratively train a global model that is then deployed to individual IoT devices ensures effective detection of intrusion types in distributed and privacy-sensitive environments. The results validate FL-Hermes as a practical and robust solution for real-world intrusion detection scenarios in IoT environments, where data privacy and distributed deployments are critical.

E. Extended Experimental Evaluation

Table X reports a comprehensive comparison of Hermes against advanced deep learning baselines on three benchmark IDS datasets. Hermes achieves competitive accuracy across all datasets, reaching 99.12% on CIC-IDS-2017, 100% on TON-IoT, and 89.17% on the challenging UNSW-NB15 dataset. While LSTM slightly outperforms Hermes on CIC-IDS-2017 in accuracy, Hermes consistently delivers substantially faster inference. On TON-IoT, both Hermes and LSTM achieve perfect classification with zero false positives. The most notable gains are observed on UNSW-NB15, where Hermes outperforms all baselines by 1.48–4.31% in accuracy while

TABLE VIII
PERFORMANCE(%) OF HERMES ON THE N-BAIoT DATASET

Index	Hermes_Sim						Hermes_MAP				
	Acc1	Recall1	Precis1	F1_1	FPR1	FPR(CL)	Acc2	Recall2	Precis2	F1_2	FPR2
0	99.98	99.99	99.99	99.99	0.03	0.28	97.30	97.31	97.30	97.30	0.82
1	99.98	99.99	99.99	99.99	0.08	0.31	96.66	96.63	96.66	96.63	0.98
2	99.86	99.97	99.77	99.87	0.28	0.31	96.19	96.19	96.24	96.09	1.21
3	99.97	99.97	99.96	99.97	0.03	0.06	96.83	96.82	96.83	96.82	0.93
4	99.94	99.99	99.92	99.95	0.15	0.26	96.00	96.02	96.00	95.97	1.22
5	99.95	99.99	99.92	99.95	0.09	0.13	97.15	97.15	97.15	97.13	0.82
6	99.98	99.97	99.99	99.98	0.01	0.52	96.69	96.70	96.69	96.68	0.99
7	99.98	99.97	99.99	99.98	0.02	0.39	95.53	95.71	95.53	95.39	1.43
8	99.97	99.98	99.99	99.99	0.07	0.68	95.56	95.75	95.56	95.43	1.46

Note: Acc2 is multi-class accuracy, while Recall2, Precis2, F1_2, and FPR2 are weighted class-wise averages.

TABLE IX
PERFORMANCE(%) OF FL-HERMES ON THE N-BAIoT DATASET

Index	FL-Hermes_Sim						FL-Hermes_MAP				
	Acc1	Recall1	Precis1	F1_1	FPR1	FPR(CL)	Acc2	Recall2	Precis2	F1_2	FPR2
0	99.60	99.55	99.65	99.60	0.35	0.21	96.29	97.30	97.28	96.28	1.40
1	99.72	99.59	99.85	99.72	0.15	0.37	97.37	95.31	96.30	97.28	1.83
2	99.55	99.42	99.69	99.55	0.31	0.42	97.32	96.70	95.30	95.38	1.17
3	99.67	99.42	99.93	99.67	0.07	0.16	96.13	96.30	97.29	97.81	0.92
4	99.59	99.33	99.85	99.59	0.15	0.25	95.80	95.39	95.19	96.31	1.91
5	99.79	99.69	99.89	99.79	0.11	0.18	97.68	96.49	97.32	96.52	1.71
6	99.97	99.95	99.99	99.99	0.01	0.36	95.28	97.30	96.29	95.71	0.86
7	99.94	99.91	99.97	99.94	0.03	0.20	96.01	95.61	97.58	97.32	1.89
8	99.92	99.90	99.95	99.92	0.05	0.46	97.32	97.42	95.30	96.12	1.15

Note: Acc2 is multi-class accuracy, while Recall2, Precis2, F1_2, and FPR2 are weighted class-wise averages.

TABLE X
PERFORMANCE (%) OF HERMES AND DEEP LEARNING BASELINES (CNN, LSTM, AND TF = TRANSFORMER) ON RECENT IDS DATASETS. VALUES ARE MEAN [95% CI] FROM 1000 BOOTSTRAP SAMPLES. TIME INDICATES AVERAGE INFERENCE TIME IN MILLISECONDS

Dataset	Model	Acc	Rec	Pre	F1	FPR	Time
CIC-IDS-2017	Hermes	99.12 [99.1, 99.14]	98.64 [98.59, 98.69]	96.94 [96.87, 97.02]	97.78 [97.74, 97.83]	0.76 [0.75, 0.78]	0.894
	CNN	96.64 [96.61, 96.68]	92.4 [92.27, 92.52]	90.73 [90.58, 90.86]	91.56 [91.45, 91.65]	2.32 [2.28, 2.36]	4.602
	LSTM	99.44 [99.42, 99.45]	98.95 [98.91, 99.0]	98.21 [98.15, 98.26]	98.58 [98.54, 98.62]	0.44 [0.43, 0.46]	7.014
	TF	98.85 [98.83, 98.87]	95.45 [95.35, 95.54]	98.65 [98.59, 98.71]	97.02 [96.96, 97.08]	0.32 [0.31, 0.33]	7.427
TON-IoT	Hermes	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	0.0 [0.0, 0.0]	1.037
	CNN	99.97 [99.97, 99.98]	99.95 [99.95, 99.96]	100.0 [100.0, 100.0]	99.98 [99.97, 99.98]	0.0 [0.0, 0.0]	4.875
	LSTM	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	0.0 [0.0, 0.0]	7.117
	TF	99.99 [99.99, 99.99]	99.99 [99.99, 99.99]	99.99 [99.99, 100.0]	99.99 [99.99, 99.99]	0.01 [0.01, 0.01]	7.876
UNSW-NB15	Hermes	89.17 [89.05, 89.31]	85.47 [85.29, 85.66]	98.41 [98.33, 98.49]	91.48 [91.38, 91.6]	2.94 [2.8, 3.08]	0.674
	CNN	84.86 [84.68, 85.0]	81.61 [81.4, 81.82]	95.5 [95.37, 95.61]	88.01 [87.86, 88.14]	8.2 [7.97, 8.41]	4.193
	LSTM	87.69 [87.55, 87.84]	83.17 [82.99, 83.38]	98.51 [98.43, 98.59]	90.19 [90.08, 90.32]	2.68 [2.54, 2.83]	7.450
	TF	87.51 [87.32, 87.7]	83.24 [83.03, 83.44]	98.12 [98.03, 98.2]	90.07 [89.93, 90.21]	3.39 [3.24, 3.57]	7.090

TABLE XI
SIGNIFICANCE TESTS: HERMES VS. BASELINES
(T-TEST / WILCOXON P-VALUES)

Dataset	Metric	vs CNN		vs LSTM		vs TF	
		t-test	Wilcoxon	t-test	Wilcoxon	t-test	Wilcoxon
CIC-IDS-2017	Acc	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
	Rec	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
	Pre	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
	F1	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
	FPR	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
TON-IoT	Acc	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
	Rec	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
	Pre	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
	F1	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
	FPR	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
UNSW-NB15	Acc	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
	Rec	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
	Pre	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
	F1	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
	FPR	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001

competing methods, highlighting its suitability for real-time intrusion detection. Statistical significance tests (Table XI) confirm that the observed performance differences are significant ($p < 0.001$). We further evaluate the training overhead of the proposed contrastive learning framework by comparing it with an MLP trained using cross-entropy loss under the same architecture and batch size. Leveraging in-mini-batch pairing and cached latent representations, Hermes incurs only a modest training time increase of $2.45\times$, $2.28\times$, and $2.59\times$ on CIC-IDS-2017, TON-IoT, and UNSW-NB15, respectively. Combined with its lightweight geometry-guided feature learning, Hermes achieves a favorable balance between accuracy, inference efficiency, and training cost, making it practical for deployment across diverse IDS scenarios.

VI. CONCLUSION

maintaining high precision (98.41%). Across all datasets, Hermes achieves approximately $5\text{--}11\times$ faster inference than

In this paper, we proposed Hermes, a machine-learning-based anomaly IDS that leverages geometric feature learning

to improve detection accuracy and reduce false positives. By learning compact and discriminative feature representations, Hermes unifies the strengths of anomaly-based and signature-based IDSs: it models normal traffic to detect deviations while incorporating known attack data to improve separability and enable attack classification. Extensive evaluations demonstrate that Hermes achieves strong detection performance and efficiency, and is suitable for deployment on resource-constrained IoT devices. To address privacy and scalability concerns, we further extended Hermes to a federated learning setting. FL-Hermes enables distributed gateways to collaboratively train IDS models without sharing raw traffic data, preserving privacy while benefiting from diverse local observations. Experimental results show that FL-Hermes maintains stable detection performance across different client scales and data distributions. While Hermes improves robustness and generalization, it does not aim to provide definitive classification for zero-day attacks, which remain a long-standing challenge in intrusion detection. Enhancing interpretability and robustness against adaptive adversaries is an important direction for future work.

REFERENCES

- [1] C. Zhang et al., "Hermes: Boosting the performance of machine-learning-based intrusion detection system through geometric feature learning," in *Proc. 25th Int. Symp. Theory, Algorithmic Found., Protocol Design Mobile Netw. Mobile Comput.*, Oct. 2024, pp. 251–260.
- [2] S. Li, S. Shi, Y. Xiao, C. Zhang, Y. T. Hou, and W. Lou, "Bijack: Breaking Bitcoin network with TCP vulnerabilities," in *Proc. Eur. Symp. Res. Comput. Secur.* Cham, Switzerland: Springer, 2023, pp. 306–326.
- [3] M. Antonakakis et al., "Understanding the Mirai botnet," in *Proc. 26th USENIX Secur. Symp.*, 2017, pp. 1093–1110.
- [4] M. Eskandari, Z. H. Janjua, M. Vecchio, and F. Antonelli, "Passban IDS: An intelligent anomaly-based intrusion detection system for IoT edge devices," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 6882–6897, Aug. 2020.
- [5] P. García-Teodoro, J. Díaz-Verdejo, G. Maciá-Fernández, and E. Vázquez, "Anomaly-based network intrusion detection: Techniques, systems and challenges," *Comput. Secur.*, vol. 28, nos. 1–2, pp. 18–28, Feb. 2009.
- [6] V. Jyothsna, V. V. R. Prasad, and K. M. Prasad, "A review of anomaly based intrusion detection systems," *Int. J. Comput. Appl.*, vol. 28, no. 7, pp. 26–35, 2011.
- [7] O. Kopuklu, J. Zheng, H. Xu, and G. Rigoll, "Driver anomaly detection: A dataset and contrastive learning approach," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2021, pp. 91–100.
- [8] Q. Yan, Y. Zheng, T. Jiang, W. Lou, and Y. T. Hou, "PeerClean: Unveiling peer-to-peer botnets through dynamic group behavior analysis," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Apr. 2015, pp. 316–324.
- [9] B. Wang, Y. Zheng, W. Lou, and Y. T. Hou, "DDoS attack protection in the era of cloud computing and software-defined networking," *Comput. Netw.*, vol. 81, pp. 308–319, Apr. 2015.
- [10] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, and J. Lloret, "Conditional variational autoencoder for prediction and feature recovery applied to intrusion detection in IoT," *Sensors*, vol. 17, no. 9, p. 1967, Aug. 2017.
- [11] H. H. Pajouh, G. Dastghaibafard, and S. Hashemi, "Two-tier network anomaly detection model: A machine learning approach," *J. Intell. Inf. Syst.*, vol. 48, no. 1, pp. 61–74, Feb. 2017.
- [12] S. Rezvy, Y. Luo, M. Petridis, A. Lasebae, and T. Zebin, "An efficient deep learning model for intrusion classification and prediction in 5G and IoT networks," in *Proc. 53rd Annu. Conf. Inf. Sci. Syst. (CISS)*, Mar. 2019, pp. 1–6.
- [13] N. Wang, Y. Chen, Y. Hu, W. Lou, and Y. T. Hou, "MANDA: On adversarial example detection for network intrusion detection system," in *Proc. IEEE Conf. Comput. Commun.*, May 2021, pp. 1–10.
- [14] N. Wang, S. Shi, Y. Chen, W. Lou, and Y. T. Hou, "FeCo: Boosting intrusion detection capability in IoT networks via contrastive learning," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 4, pp. 4215–4230, Jul. 2025.
- [15] C. Zhang, N. Wang, S. Shi, C. Du, W. Lou, and Y. T. Hou, "MINDFL: Mitigating the impact of imbalanced and noisy-labeled data in federated learning with quality and fairness-aware client selection," in *Proc. MILCOM - IEEE Mil. Commun. Conf. (MILCOM)*, Oct. 2023, pp. 331–338.
- [16] S. Anwar et al., "From intrusion detection to an intrusion response system: Fundamentals, requirements, and future directions," *Algorithms*, vol. 10, no. 2, p. 39, Mar. 2017.
- [17] M. Bashendy, A. Tantawy, and A. Erradi, "Intrusion response systems for cyber-physical systems: A comprehensive survey," *Comput. Secur.*, vol. 124, Jan. 2023, Art. no. 102984.
- [18] G. Dayanandam, T. Rao, D. B. Babu, and S. N. Durga, "DDoS attacks-analysis and prevention," in *Proc. 5th Innov. Comput. Sci. Eng. Cham*, Switzerland: Springer, 2019, pp. 1–10.
- [19] M. Yao, J. Fuller, R. P. Kasturi, S. Agarwal, A. K. Sikder, and B. Saltaformaggio, "Hiding in plain sight: An empirical study of Web application abuse in malware," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 6115–6132.
- [20] R. Canzanese, "Cloud as a malware infiltration channel," *Comput. Fraud Secur.*, vol. 2023, no. 4, Apr. 2023.
- [21] H. Hindy, R. Atkinson, C. Tachtatzis, J.-N. Colin, E. Bayne, and X. Bellekens, "Utilising deep learning techniques for effective zero-day attack detection," *Electronics*, vol. 9, no. 10, p. 1684, Oct. 2020.
- [22] S. I. Popoola, R. Ande, B. Adebisi, G. Gui, M. Hammoudeh, and O. Jogunola, "Federated deep learning for zero-day botnet attack detection in IoT-edge devices," *IEEE Internet Things J.*, vol. 9, no. 5, pp. 3930–3944, Mar. 2022.
- [23] J. Holland, P. Schmitt, N. Feamster, and P. Mittal, "New directions in automated traffic analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3366–3383.
- [24] G. D'Angelo and F. Palmieri, "Network traffic classification using deep convolutional recurrent autoencoder neural networks for spatial-temporal features extraction," *J. Netw. Comput. Appl.*, vol. 173, Jan. 2021, Art. no. 102890.
- [25] Y. Yue, X. Chen, Z. Han, X. Zeng, and Y. Zhu, "Contrastive learning enhanced intrusion detection," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 4, pp. 4232–4247, Dec. 2022.
- [26] X. Zhang et al., "AOC-IDS: Autonomous online framework with contrastive learning for intrusion detection," in *Proc. IEEE Conf. Comput. Commun.*, May 2024, pp. 581–590.
- [27] X. Xu and L. Zheng, "Neural feature learning in function space," *J. Mach. Learn. Res.*, vol. 25, no. 142, pp. 1–76, 2024.
- [28] X. Xu and S.-L. Huang, "Maximal correlation regression," *IEEE Access*, vol. 8, pp. 26591–26601, 2020.
- [29] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, vol. 54, Fort Lauderdale, FL, USA, 2017, pp. 1273–1282.
- [30] P. Kairouz and H. B. McMahan, "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, nos. 1–2, pp. 1–210, Jun. 2021.
- [31] N. H. Tran, W. Bao, A. Zomaya, M. N. H. Nguyen, and C. S. Hong, "Federated learning over wireless networks: Optimization model design and analysis," in *Proc. IEEE Conf. Comput. Commun.*, Apr. 2019, pp. 1387–1395.
- [32] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl.*, Jul. 2009, pp. 1–6.
- [33] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018.
- [34] S. Rathore and J. H. Park, "Semi-supervised learning based distributed attack detection framework for IoT," *Appl. Soft Comput.*, vol. 72, pp. 79–89, Nov. 2018.
- [35] Y. Yang, K. Zheng, C. Wu, and Y. Yang, "Improving the classification effectiveness of intrusion detection by using improved conditional variational AutoEncoder and deep neural network," *Sensors*, vol. 19, no. 11, p. 2528, Jun. 2019.
- [36] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS)*, Nov. 2015, pp. 1–6.
- [37] H. H. Pajouh, R. Javidan, R. Khayami, A. Dehghantanha, and K.-K.-R. Choo, "A two-layer dimension reduction and two-tier classification model for anomaly-based intrusion detection in IoT backbone networks," *IEEE Trans. Emerg. Topics Comput.*, vol. 7, no. 2, pp. 314–323, Apr. 2019.

- [38] N. Moustafa, B. Turnbull, and K.-K.-R. Choo, "An ensemble intrusion detection technique based on proposed statistical flow features for protecting network traffic of Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4815–4830, Jun. 2019.
- [39] R. Alshammari and A. N. Zincir-Heywood, "Can encrypted traffic be identified without port numbers, IP addresses and payload inspection?," *Comput. Netw.*, vol. 55, no. 6, pp. 1326–1350, Apr. 2011.
- [40] C. Kolias, G. Kambourakis, A. Stavrou, and S. Gritzalis, "Intrusion detection in 802.11 networks: Empirical evaluation of threats and a public dataset," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 1, pp. 184–208, 1st Quart., 2016.
- [41] Y. Zhang et al., "Efficient and intelligent attack detection in software defined IoT networks," in *Proc. IEEE Int. Conf. Embedded Softw. Syst. (ICESS)*, Jul. 2020, pp. 1–9.
- [42] R. Fontes, S. Afzal, M. Brito, M. Santos, and C. Esteve Rothenberg, "Mininet-WiFi: Emulating software-defined wireless networks," in *Proc. 11th Int. Conf. Netw. Service Manag.*, Nov. 2015, pp. 1–16.
- [43] Y. Fan, Y. Li, M. Zhan, H. Cui, and Y. Zhang, "IoTDefender: A federated transfer learning intrusion detection framework for 5G IoT," in *Proc. IEEE 14th Int. Conf. Big Data Sci. Eng. (BigDataSE)*, Guangzhou, China, Dec. 2020, pp. 88–95.
- [44] I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 108–116.
- [45] N. Ravi and S. M. Shalinie, "Semisupervised-learning-based security to detect and mitigate intrusions in IoT network," *IEEE Internet Things J.*, vol. 7, no. 11, pp. 11041–11052, Nov. 2020.
- [46] Y. Meidan et al., "N-BaIoT—Network-based detection of IoT botnet attacks using deep autoencoders," *IEEE Pervasive Comput.*, vol. 17, no. 3, pp. 12–22, Jul. 2018.
- [47] Q. Yan et al., "SpecMonitor: Toward efficient passive traffic monitoring for cognitive radio networks," *IEEE Trans. Wireless Commun.*, vol. 13, no. 10, pp. 5893–5905, Oct. 2014.
- [48] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 1285–1298.
- [49] T. D. Nguyen et al., "DioT: A federated self-learning anomaly detection system for IoT," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Aug. 2019, pp. 756–767.
- [50] N. Chaabouni, M. Mosbah, A. Zemmari, C. Sauvignac, and P. Faruki, "Network intrusion detection for IoT security based on learning techniques," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2671–2701, 3rd Quart., 2019.
- [51] M. A. Al-Garadi, A. Mohamed, A. K. Al-Ali, X. Du, I. Ali, and M. Guizani, "A survey of machine and deep learning methods for Internet of Things (IoT) security," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 3, pp. 1646–1685, 3rd Quart., 2020.
- [52] B. Claise, "Cisco systems netflow services export version 9," *RFC*, vol. 3954, pp. 1–33, 2004. [Online]. Available: <https://api.semanticscholar.org/CorpusID:6125326>
- [53] (2026). *CICFlowmeter: A Network Traffic Flow Generator and Analyzer*. [Online]. Available: <https://www.unb.ca/cic/research/applications.html#CICFlowMeter>
- [54] N. Wang, C. Zhang, Y. Xiao, Y. Chen, W. Lou, and Y. T. Hou, "FLARE: Defending federated learning against model poisoning attacks via latent space representations," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 3, pp. 2607–2623, May 2025.
- [55] I. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proc. Int. Conf. Learn. Represent.*, 2014.
- [56] R. Fotuhi, F. Shams Aliee, and B. Farahani, "Decentralized and robust privacy-preserving model using blockchain-enabled federated deep learning in intelligent enterprises," *Appl. Soft Comput.*, vol. 161, Aug. 2024, Art. no. 111764.
- [57] R. Fotuhi, F. Shams Aliee, and B. Farahani, "A lightweight and secure deep learning model for privacy-preserving federated learning in intelligent enterprises," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 31988–31998, Oct. 2024.
- [58] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. 37th Int. Conf. Mach. Learn.*, vol. 119, pp. 1597–1607.
- [59] X. Xu and L. Zheng, "Multivariate feature extraction," in *Proc. 58th Annu. Allerton Conf. Commun., Control, Comput. (Allerton)*, Sep. 2022, pp. 1–8.
- [60] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, Nov. 2011.
- [61] C. Li, X. Zeng, M. Zhang, and Z. Cao, "PyramidFL: A fine-grained client selection framework for efficient federated learning," in *Proc. 28th Annu. Int. Conf. Mobile Comput. Netw.*, Oct. 2022, pp. 158–171.
- [62] Y. Deng et al., "AUCTION: Automated and quality-aware client selection framework for efficient federated learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 8, pp. 1996–2009, Aug. 2022.



Chaoyu Zhang (Student Member, IEEE) received the Bachelor of Science degree in electrical engineering from Beijing University of Posts and Telecommunications, Beijing, China, in 2018, and the Master of Science degree in computer science from Arkansas State University, Arkansas, USA, in 2021. He is currently pursuing the Ph.D. degree in computer science with Virginia Tech, USA, under the supervision of Prof. Wenjing Lou. His research focuses on machine learning security, network security, blockchain technology, and high-performance computing. He was a recipient of the Best Paper Award at IEEE CNS 2024.



Shanghao Shi (Member, IEEE) received the B.S. degree in telecommunication engineering from Beijing University of Posts and Telecommunications in 2019 and the Ph.D. degree in computer science from Virginia Tech in 2025. He is currently a Post-Doctoral Research Associate of computer science with Washington University in St. Louis. His research focuses on network security and information privacy, with particular interests in machine learning security and privacy, wireless security, and CPS/IoT security.



Ning Wang received the B.E. degree in communication engineering and the M.S. degree in electronics and communication engineering from Beijing University of Posts and Telecommunications, Beijing, China, in 2015 and 2018, respectively, and the Ph.D. degree in computer engineering from Virginia Tech in 2023. She is currently an Assistant Professor with the Department of Computer Science and Engineering, University of South Florida. Her current research interests include federated learning, anomaly detection, adversarial machine learning, differential privacy, and LLM applications in cybersecurity.



Xiangxiang Xu (Member, IEEE) received the B.Eng. and Ph.D. degrees in electronic engineering from Tsinghua University, Beijing, China, in 2014 and 2020, respectively. He is currently an Assistant Professor of computer science with the University of Rochester. Prior to joining the University of Rochester, he was a Post-Doctoral Associate with the Department of EECS, MIT. His research focuses on information theory, statistical learning, representation learning, and their applications in understanding and developing artificial intelligence and machine learning algorithms. He was a recipient of the IEEE PES Student Prize Paper Award in Honor of T. Burke Hayes and the Information Theory and Applications (ITA) Workshop Sand Award.



Shaoyu Li (Student Member, IEEE) received the B.S. degree from the College of Cyber Science, Nankai University, and the M.S. degree from the College of Science and Engineering, University of Glasgow. He is currently pursuing the Ph.D. degree with the Department of Computer Science, Virginia Tech, under the supervision of Prof. Wenjing Lou. His research interests include network security and blockchain.



Lizhong Zheng (Fellow, IEEE) received the B.S. and M.S. degrees from the Department of Electronic Engineering, Tsinghua University, and the Ph.D. degree from the Department of Electrical Engineering and Computer Science, UC Berkeley. Since 2002, he has been with MIT, where he is currently a Professor of electrical engineering and computer science. His current research interests include statistics, information theory, and their applications in data science. He received the NSF CAREER Award, the AFOSR Young Investigator Award, and the IEEE

Information Theory Society Paper Award. He is an Area Editor of IEEE TRANSACTIONS ON INFORMATION THEORY.



Randy Marchany (Member, IEEE) has been involved in the computer industry since 1972. He is currently the Virginia Tech Information Technology Security Officer and the Director of Virginia Tech IT Security Laboratory. He is the co-holder of three cybersecurity patents on moving target IPV6 defense and IDS via power monitoring. His research interests are in network security and forensics. He is a member of the Center for Internet Security (CIS) working group that created version 8 of the CIS Security Controls. He is a Former Member of the

Research, Education, Networking, Information Sharing, and Analysis Center (REN-ISAC) Board. He is an Executive Committee Member of Virginia Cyber Range. He is one of the founders of Virginia Alliance for Secure Computing and Networking, a consortium of security practitioners and researchers from the major universities in Virginia. He received the 2021 SANS Difference Maker Award. He was a recipient of the 2016 Shirley C. Payne IT Security Advancement Award, the 2000 SANS Institute's Security Technology Leadership Award, the 2003 VA Governor's Technology Silver Award, a member of the team that won the EDUCAUSE Excellence in Information Technology Solutions Award in 2005, and a member of the Virginia Cyber Range Team that won the 2017 Virginia Governor's Technology Award for innovative use of technology in education.



Mark Gardner (Member, IEEE) is currently the Network Research Manager of advanced research computing with the Division of Information Technology, Virginia Tech. His research interests include network monitoring and measurement, cybersecurity, virtualization, high-performance networking, high-performance storage and file systems, research computing, operating systems, and programming languages, including interpreters and compilers.



Y. Thomas Hou (Fellow, IEEE) received the Ph.D. degree from the NYU Tandon School of Engineering in 1998. He is currently the Bradley Distinguished Professor of electrical and computer engineering with Virginia Tech, Blacksburg, VA, USA, where he joined in 2002. He has published over 400 papers in IEEE/ACM journals and conferences. He holds six U.S. patents. He has authored/co-authored two graduate textbooks, such as *Applied Optimization Methods for Wireless Networks* (Cambridge University Press, 2014) and

Cognitive Radio Communications and Networks: Principles and Practices (Academic Press/Elsevier, 2009). His current research focuses on developing real-time optimal solutions to complex science and engineering problems arising from wireless and mobile networks. He is also interested in wireless security. He was named an IEEE Fellow for contributions to the modeling and optimization of wireless networks. His papers were recognized by 12 best paper awards from IEEE and ACM, including the IEEE INFOCOM Test of Time Paper Award in 2023. He was the Steering Committee Chair of the IEEE INFOCOM Conference and a member of the IEEE Communications Society Board of Governors. He served on the editorial boards of several IEEE and ACM transactions and journals. He was a Distinguished Lecturer of the IEEE Communications Society.



Wenjing Lou (Fellow, IEEE) is the W. C. English Endowed Professor of computer science with Virginia Tech. Her research interests cover many topics in the cybersecurity field, with her current research interests focusing on wireless network security, trustworthy AI, blockchain, and security and privacy problems in the Internet of Things (IoT) systems. She is a Steering Committee Member of IEEE INFOCOM and IEEE TRANSACTIONS ON MOBILE COMPUTING. She served as the Program Director for U.S. National Science Foundation (NSF) from

2014 to 2017. She is a Highly Cited Researcher by the Web of Science Group. She received the Virginia Tech Alumni Award for Research Excellence in 2018 and the INFOCOM Test-of-Time Paper Award in 2020. She was the TPC Chair of IEEE INFOCOM 2019 and ACM WiSec 2020. She was the Steering Committee Chair of the IEEE CNS Conference from 2013 to 2020.